



Protocol Audit Report

Version 1.0

Cyfrin.io

May 12, 2024

Puppy Raffle Audit Report

Freeman

May 12, 2024

Contents

1	Protocol Summary	3
2	Disclaimer	3
3	Risk Classification	3
4	Audit Details	3
4.1	Scope	4
4.2	Roles	4
5	Executive Summary	4
5.1	Issues Found	4
6	Findings	4
6.1	High	4
6.1.1	[H-1] Reentrancy Attack in <code>PuppyRaffle::refund</code> Allows En- trant to Drain Contract Balance	4
6.1.2	[H-2] Weak Randomness in <code>PuppyRaffle::selectWinner</code> Al- lows Anyone to Choose Winner	5
6.1.3	[H-3] Integer Overflow of <code>PuppyRaffle::totalFees</code> Loses Fees	5
6.1.4	[H-4] Malicious Winner Can Forever Halt the Raffle	6
6.2	Medium	6
6.2.1	[M-1] Looping Through Players Array in <code>PuppyRaffle::enterRaffle</code> is a DoS Vector	6
6.2.2	[M-2] Balance Check in <code>PuppyRaffle::withdrawFees</code> Enables Griefing	6
6.2.3	[M-3] Unsafe Cast of <code>PuppyRaffle::fee</code> Loses Fees	7
6.2.4	[M-4] Smart Contract Wallet Winners Without <code>receive/fallback</code> Block New Contests	7
6.3	Informational / Non-Critical	7
6.3.1	[I-1] Floating Pragmas	7
6.3.2	[I-2] Magic Numbers	7
6.3.3	[I-3] Test Coverage	7
6.3.4	[I-4] Zero Address Validation	7

6.3.5	[I-5] Unused _isActivePlayer Function	7
6.3.6	[I-6] Unchanged Variables Should Be Constant or Immutable	7
6.3.7	[I-7] Potentially Erroneous Active Player Index	8
6.3.8	[I-8] Zero Address Considered Active Player	8
6.4	Gas	8
6.4.1	[G-1] Storage Variables in a Loop Should Be Cached	8
6.4.2	[G-2] Unchanged State Variables Should Be Constant or Immutable	8

Prepared by

Freeman

Lead Auditors

- Freeman

1 Protocol Summary

This project enables participants to enter a raffle to win a unique dog NFT. The protocol operates as follows:

- The **enterRaffle** function accepts a list of participant addresses, allowing multiple entries by the same address or group entries.
- Duplicate addresses are prohibited.
- Participants can request a refund of their ticket value via the **refund** function.
- Every fixed interval, the raffle selects a winner who receives a random puppy NFT.
- The protocol owner designates a **feeAddress** to collect a portion of the funds, with the remainder awarded to the winner.

2 Disclaimer

The Chain Sentinel team has made every effort to identify vulnerabilities within the given timeframe but assumes no responsibility for the findings in this document. This security audit is not an endorsement of the underlying business or product. The review was time-boxed and focused solely on the security aspects of the Solidity implementation.

3 Risk Classification

Likelihood	Impact		
	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Table 1: Risk classification matrix as per [CodeHawks](#).

4 Audit Details

Commit Hash: e30d199697bbc822b646d76533b66b7d529b8ef5

4.1 Scope

The audit covers the following file:

- `./src/PuppyRaffle.sol`

4.2 Roles

- **Owner:** The protocol deployer, authorized to update the `feeAddress` via the `changeFeeAddress` function.
- **Player:** Raffle participants who can enter via `enterRaffle` and request refunds through `refund`.

5 Executive Summary

The audit of the Puppy Raffle codebase was engaging, revealing deliberate vulnerabilities for educational purposes.

5.1 Issues Found

Severity	Number of Issues
High	4
Medium	4
Low	0
Informational	8
Gas	2
Total	18

6 Findings

6.1 High

6.1.1 [H-1] Reentrancy Attack in `PuppyRaffle::refund` Allows Entrant to Drain Contract Balance

Description: The `refund` function is vulnerable to reentrancy due to non-compliance with the Checks-Effects-Interactions pattern. It sends funds before updating the `players` array, allowing a malicious participant to repeatedly call `refund` via a `fallback` or `receive` function.

```
1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the
      player can refund");
4     require(playerAddress != address(0), "PuppyRaffle: Player
      already refunded, or is not active");
```

```

5 payable(msg.sender).sendValue(entranceFee);
6 players[playerIndex] = address(0);
7 emit RaffleRefunded(playerAddress);
8 }

```

Impact: A malicious participant could drain all fees collected from raffle entrants.

Proof of Concept:

1. A user enters the raffle.
2. An attacker deploys a contract with a **fallback** function that re-calls **refund**.
3. The attacker enters the raffle and calls **refund**, draining the contract balance.

Recommended Mitigation: Update the **players** array and emit events before the external call.

```

1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the
4         player can refund");
5     require(playerAddress != address(0), "PuppyRaffle: Player
6         already refunded, or is not active");
7     players[playerIndex] = address(0);
8     emit RaffleRefunded(playerAddress);
9     (bool success,) = msg.sender.call{value: entranceFee}("");
10    require(success, "PuppyRaffle: Failed to refund player");
11 }

```

6.1.2 [H-2] Weak Randomness in **PuppyRaffle::selectWinner** Allows Anyone to Choose Winner

Description: The **selectWinner** function uses predictable inputs (**msg.sender**, **block.timestamp**, **block.difficulty**) for randomness, enabling manipulation.

Impact: Malicious users can predetermine the winner, undermining the raffle's fairness.

Proof of Concept:

- Validators can predict **block.timestamp** and **block.difficulty** (now **pre-vrandao**).
- Users can manipulate **msg.sender** to influence the winner index.

Recommended Mitigation: Use a secure oracle like [Chainlink VRF](#) for randomness.

6.1.3 [H-3] Integer Overflow of **PuppyRaffle::totalFees** Loses Fees

Description: The **totalFees** variable, a **uint64**, can overflow in Solidity < 0.8.0, causing fees to be lost.

Impact: Fees may become uncollectible, remaining stuck in the contract.

Proof of Concept:

1. Conclude a raffle with 4 players, collecting fees.
2. Enter 89 additional players and conclude the raffle.
3. `totalFees` overflows, resulting in an incorrect value.
4. The `withdrawFees` balance check fails, blocking withdrawals.

Recommended Mitigation:

- Use Solidity ^{0.8.18} to prevent overflows. Change `totalFees` to `uint256`.
- Remove the balance check in `withdrawFees`.

6.1.4 [H-4] Malicious Winner Can Forever Halt the Raffle

Description: The `selectWinner` function may fail if the winner is a smart contract without a payable `receive/fallback` or `onERC721Received` hook, preventing prize distribution and raffle reset.

Impact: The raffle could be permanently halted.

Proof of Concept:

- A smart contract without `onERC721Received` enters and wins.
- `selectWinner` reverts, blocking the raffle reset.

Recommended Mitigation: Implement pull-based payments, requiring winners to claim prizes.

6.2 Medium

6.2.1 [M-1] Looping Through Players Array in `PuppyRaffle::enterRaffle` is a DoS Vector

Description: The duplicate check loop in `enterRaffle` increases gas costs as the `players` array grows.

Impact: Higher gas costs for later entrants and potential front-running opportunities.

Recommended Mitigation:

- Allow duplicates, as users can create new wallets.
- Use a mapping to track entries by raffle ID.

6.2.2 [M-2] Balance Check in `PuppyRaffle::withdrawFees` Enables Griefing

Description: The balance check in `withdrawFees` can be exploited by sending ETH via `selfdestruct`, blocking withdrawals.

Impact: Prevents `feeAddress` from withdrawing fees.

Recommended Mitigation: Remove the balance check.

6.2.3 [M-3] Unsafe Cast of **PuppyRaffle::fee** Loses Fees

Description: Casting **fee** to **uint64** in **selectWinner** can truncate values exceeding **uint64.max**.

Impact: Incorrect fee collection, leaving funds stuck.

Recommended mapping: Use **uint256** for **totalFees** and remove casting.

6.2.4 [M-4] Smart Contract Wallet Winners Without **receive/fallback** Block New Contests

Description: If a winner is a smart contract without **receive/fallback**, **selectWinner** may revert, delaying raffle reset.

Impact: Hinders new raffle starts and payout distribution.

Recommended Mitigation: Use a pull-based payment system.

6.3 Informational / Non-Critical

6.3.1 [I-1] Floating Pragmas

Description: Using floating pragmas risks deployment with untested Solidity versions.

Recommended Mitigation: Lock the pragma to **0.7.6**.

6.3.2 [I-2] Magic Numbers

Description: Hardcoded numbers reduce code readability.

Recommended Mitigation: Use constants for percentages.

6.3.3 [I-3] Test Coverage

Description: Test coverage is below 90%, indicating untested code paths.

Recommended Mitigation: Increase coverage, especially for branches.

6.3.4 [I-4] Zero Address Validation

Description: Lack of zero-address checks for **feeAddress** risks fee loss.

Recommended Mitigation: Add zero-address validation.

6.3.5 [I-5] Unused **_isActivePlayer** Function

Description: The **_isActivePlayer** function is unused.

Recommended Mitigation: Remove the function.

6.3.6 [I-6] Unchanged Variables Should Be Constant or Immutable

Description: Variables like **raffleDuration** and image URIs are not marked as **constant** or **immutable**.

Recommended Mitigation: Declare as **constant** or **immutable**.

6.3.7 [I-7] Potentially Erroneous Active Player Index

Description: `getActivePlayerIndex` may return 0 for inactive players, causing confusion.

Recommended Mitigation: Return a high number for inactive players.

6.3.8 [I-8] Zero Address Considered Active Player

Description: `getActivePlayerIndex` may treat zero addresses as active post-refunded.

Recommended Mitigation: Skip zero addresses in `getActivePlayerIndex` and prevent their entry.

6.4 Gas

6.4.1 [G-1] Storage Variables in a Loop Should Be Cached

Description: Repeatedly accessing `players.length` in loops increases gas costs.

Recommended Mitigation: Cache `players.length`.

6.4.2 [G-2] Unchanged State Variables Should Be Constant or Immutable

Description: Unmarked `constant/immutable` variables increase gas costs.

Recommended Mitigation: Declare appropriate variables as `constant` or `immutable`.